

Approximation Algorithms

Performance Ratios, Vertex Cover, and Traveling Salesperson

1 Why Approximation Algorithms?

Central Idea

Many important optimization problems are **NP-complete** or **NP-hard**. For such problems, we usually do not expect a polynomial-time algorithm that always returns the exact optimum. An **approximation algorithm** runs in polynomial time and returns a solution that is provably close to optimal.

When a problem is computationally hard, there are several common ways to still get useful results:

- use brute force when the input is small;
- isolate special cases that are polynomial-time solvable;
- use design techniques such as divide-and-conquer, greedy algorithms, or dynamic programming when they apply;
- accept a near-optimal solution with a guaranteed quality bound.

Approximation algorithms are useful when a solution that is “good enough” is valuable and exact optimality is too expensive.

2 Performance Ratios

Notation

For an optimization problem instance:

$$C^* = \text{cost of an optimal solution}$$

and

$$C = \text{cost of the solution returned by the approximation algorithm.}$$

For graph problems, the input size is often written as $n = |V|$ or $n = |V| + |E|$, depending on context.

The meaning of “close to optimal” depends on whether the problem is a minimization problem or a maximization problem.

Problem Type	Relationship Between C and C^*	Ratio Measured
Minimization	$0 < C^* \leq C$	$\frac{C}{C^*}$, the factor by which the algorithm is worse than optimum.
Maximization	$0 < C \leq C^*$	$\frac{C^*}{C}$, the factor by which the optimum is better than the algorithm.

Approximation Ratio

An algorithm has **approximation ratio** $\rho(n)$ if, for every input of size n , its cost is within a factor of $\rho(n)$ of the optimum:

$$\max\left(\frac{C}{C^*}, \frac{C^*}{C}\right) \leq \rho(n).$$

For a minimization problem this becomes:

$$C \leq \rho(n)C^*.$$

For a maximization problem this becomes:

$$C \geq \frac{C^*}{\rho(n)}.$$

How to Read the Ratio

A smaller approximation ratio is better. A ratio of 1 means the algorithm is exact. A 2-approximation for a minimization problem always returns a solution whose cost is at most twice the optimum.

3 Approximation Schemes

Approximation Scheme

An **approximation scheme** is an algorithm that takes both:

- an input instance, and
- an accuracy parameter $\varepsilon > 0$,

and returns a solution with approximation ratio $1 + \varepsilon$ for a minimization problem.

Polynomial-Time Approximation Scheme

A **polynomial-time approximation scheme** (PTAS) is an approximation scheme whose running time is polynomial in the input size n for every fixed value of $\varepsilon > 0$.

The phrase “for every fixed ε ” is important. A PTAS may be polynomial in n but very expensive as ε becomes small. For example, $n^{1/\varepsilon}$ is polynomial in n for fixed ε , but it becomes impractical when ε is small.

4 How Approximation Algorithms Are Designed

Approximation algorithms usually exploit special structure in the problem. Common techniques include:

- **Greedy choice:** repeatedly choose a locally useful object.
- **Linear programming:** solve a relaxed version, then round the fractional answer.
- **Dynamic programming:** carefully round or scale values to get a faster approximate solution.
- **Randomization:** choose random assignments and prove a good expected value.

Proof Strategy

To prove an approximation ratio, we often do not know C^* exactly. Instead, we prove a lower bound on C^* for minimization problems, or an upper bound on C^* for maximization problems, and compare the algorithm's solution against that bound.

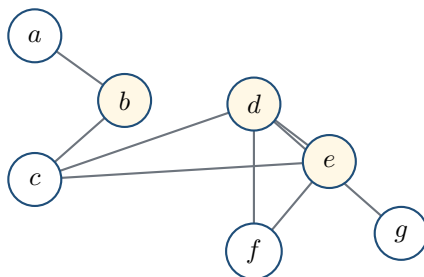
5 The Vertex-Cover Problem

Definition: Vertex Cover

Let $G = (V, E)$ be an undirected graph. A **vertex cover** is a set $C \subseteq V$ such that every edge has at least one endpoint in C .

$$\forall (u, v) \in E, \quad u \in C \text{ or } v \in C.$$

The VERTEX-COVER problem asks for a vertex cover of minimum size. The size of a vertex cover is the number of vertices in it. The decision version of VERTEX-COVER is **NP-complete**, so the optimization version is **NP-hard**.



For the graph above, one optimal vertex cover is

$$C^* = \{b, d, e\}.$$

It covers every edge:

$$(a, b), (b, c), (c, d), (c, e), (d, e), (d, f), (d, g), (e, f).$$

Thus the optimal cost is $|C^*| = 3$.

6 A Greedy 2-Approximation for Vertex Cover

Approx-Vertex-Cover

- 1 $C \leftarrow \emptyset$
- 2 $E' \leftarrow E$
- 3 While $E' \neq \emptyset$, choose any edge $(u, v) \in E'$
- 4 $C \leftarrow C \cup \{u, v\}$
- 5 Remove from E' every edge incident on u or v
- 6 Return C

The algorithm repeatedly picks an uncovered edge and places both of its endpoints into the cover. Once those endpoints are chosen, every edge touching either one is already covered, so those incident edges can be discarded.

Key Observation

The edges chosen by APPROX-VERTEX-COVER form a matching: no two chosen edges share an endpoint. This is true because after an edge (u, v) is chosen, all edges incident on u or v are removed.

6.1 Example Runs

The output depends on which uncovered edge is selected at each step.

Run	Chosen Edges	Returned Vertex Cover
Run 1	$(a, b), (c, d), (e, f)$	$C = \{a, b, c, d, e, f\}$, so $ C = 6$.
Run 2	$(d, e), (b, c)$	$C = \{b, c, d, e\}$, so $ C = 4$.

Both outputs are valid vertex covers, but neither is required to be optimal. In the first run,

$$\frac{|C|}{|C^*|} = \frac{6}{3} = 2.$$

6.2 Running Time

Using adjacency lists and marking deleted edges, each vertex and edge can be processed a constant number of times. Therefore:

$$T(V, E) = \mathcal{O}(|V| + |E|).$$

Theorem 1. APPROX-VERTEX-COVER is a polynomial-time 2-approximation algorithm for VERTEX-COVER.

Proof. The algorithm runs in $\mathcal{O}(|V| + |E|)$ time, so it is polynomial.

Let A be the set of edges selected by the algorithm. No two edges in A share an endpoint, so A is a matching. Any vertex cover, including an optimal vertex cover C^* , must contain at least one endpoint of every edge in A . Because the edges in A do not share endpoints, one chosen vertex cannot cover two different edges of A . Therefore:

$$|C^*| \geq |A|.$$

For each edge in A , the algorithm adds exactly two endpoints to C . Hence:

$$|C| = 2|A|.$$

Combining the two inequalities gives:

$$|C| = 2|A| \leq 2|C^*|.$$

Thus the returned cover has size at most twice the optimal size. $\square$

7 The Traveling-Salesperson Problem

Definition: TSP

In the traveling-salesperson problem, we are given a complete undirected graph $G = (V, E)$ and a nonnegative integer cost $c(u, v)$ for every edge. The goal is to find a Hamiltonian cycle of minimum total cost.

A Hamiltonian cycle visits every vertex exactly once and returns to the starting vertex. The decision version of TSP is **NP-complete**, and the optimization version is **NP-hard**.

8 Triangle Inequality

Metric Cost Function

The cost function satisfies the **triangle inequality** if, for all vertices $u, v, w \in V$,

$$c(u, w) \leq c(u, v) + c(v, w).$$

The triangle inequality says that going directly from u to w is never more expensive than detouring through v . A TSP instance with this property is often called **metric TSP**.

Why Triangle Inequality Matters

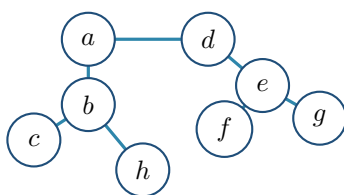
The metric TSP approximation algorithm works by taking a walk that may repeat vertices and then shortcutting repeated vertices. The triangle inequality is exactly what guarantees that shortcutting does not increase the cost.

9 A 2-Approximation for Metric TSP

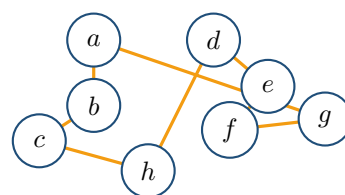
Approx-TSP-Tour(G, c)

- 1 Select a root vertex $r \in V$
- 2 Compute a minimum spanning tree T of G , rooted at r
- 3 Let L be the list of vertices in a preorder walk of T
- 4 Return the Hamiltonian cycle H that visits vertices in the order L

The preorder walk visits a vertex when it is first reached in a depth-first traversal of the tree. If the tree walk returns to a vertex later, the algorithm does not list it again in the final tour.



Minimum spanning tree T



Shortcut Hamiltonian tour H

For the tree shown above, one full tree walk is

$$a, b, c, b, h, b, a, d, e, f, e, g, e, d, a.$$

After deleting repeated vertices, the preorder list is

$$L = (a, b, c, h, d, e, f, g),$$

so the algorithm returns the tour

$$a \rightarrow b \rightarrow c \rightarrow h \rightarrow d \rightarrow e \rightarrow f \rightarrow g \rightarrow a.$$

9.1 Running Time

For a complete graph represented by an adjacency matrix, Prim's MST algorithm runs in $\mathcal{O}(|V|^2)$ time. The preorder traversal and shortcut construction are linear after the MST is built. Therefore:

$$T(V) = \mathcal{O}(|V|^2).$$

Theorem 2. *APPROX-TSP-TOUR is a polynomial-time 2-approximation algorithm for TSP with triangle inequality.*

Proof. Let H^* be an optimal TSP tour. If we remove any one edge from H^* , the remaining edges form a spanning tree. Since T is a minimum spanning tree,

$$\text{cost}(T) \leq \text{cost}(H^*).$$

Now create a full walk W by walking around the tree T . This full walk traverses every edge of T exactly twice, once in each direction. Therefore:

$$\text{cost}(W) = 2 \text{cost}(T) \leq 2 \text{cost}(H^*).$$

The full walk may visit some vertices more than once. The algorithm converts W into a Hamiltonian tour H by shortcutting repeated vertices. Because the cost function satisfies the triangle inequality, replacing a path $u \rightarrow v \rightarrow w$ by the direct edge $u \rightarrow w$ cannot increase the cost:

$$c(u, w) \leq c(u, v) + c(v, w).$$

Applying this shortcut repeatedly gives:

$$\text{cost}(H) \leq \text{cost}(W).$$

Thus:

$$\text{cost}(H) \leq \text{cost}(W) \leq 2 \text{cost}(H^*).$$

So the algorithm returns a tour whose cost is at most twice the optimal tour. $\square$

10 TSP Without Triangle Inequality

Negative Result

If $\mathbf{P} \neq \mathbf{NP}$, then for any constant $\rho \geq 1$, there is no polynomial-time approximation algorithm with approximation ratio ρ for the general TSP problem.

The triangle inequality is not just a small technical condition. Without it, shortcutting can become arbitrarily expensive, and even obtaining any fixed constant-factor guarantee would be enough to solve an **NP-complete** problem.

10.1 Proof Idea

Assume, for contradiction, that a polynomial-time ρ -approximation algorithm exists for general TSP, where $\rho \geq 1$ is a constant.

Given an undirected graph $G = (V, E)$ with $n = |V|$, build a complete graph G' on the same vertices:

$$c(u, v) = \begin{cases} 1, & \text{if } (u, v) \in E, \\ \lceil \rho n \rceil + 1, & \text{if } (u, v) \notin E. \end{cases}$$

If G has a Hamiltonian cycle, then G' has a TSP tour of cost exactly n . The approximation algorithm must return a tour of cost at most ρn .

If G has no Hamiltonian cycle, then every tour in G' must use at least one edge that was not in E . Therefore every tour has cost greater than ρn .

So the approximation algorithm would let us decide whether G has a Hamiltonian cycle by checking whether the returned cost is at most ρn . That would solve the Hamiltonian-cycle problem in polynomial time. Since Hamiltonian cycle is **NP-complete**, this would imply **P = NP**.

11 Key Ideas

Idea	Meaning
Approximation algorithm	Polynomial-time algorithm that returns a provably near-optimal solution.
Approximation ratio	Worst-case factor comparing the algorithm's cost to the optimal cost.
2-approximation	Always within a factor of 2 of optimum.
PTAS	Takes $\varepsilon > 0$ and gives ratio $1 + \varepsilon$ in polynomial time for fixed ε .
Approx-Vertex-Cover	Greedy algorithm that picks an uncovered edge and adds both endpoints.
Vertex-cover proof	Chosen edges form a matching; every cover needs at least one endpoint per chosen edge.
Metric TSP	TSP whose costs satisfy the triangle inequality.
Approx-TSP-Tour	Build an MST, walk around it, then shortcut repeated vertices.
General TSP hardness	Without triangle inequality, no constant-factor approximation exists unless P = NP .

12 Quick Self-Check

Review Questions

1. Why are approximation algorithms useful for **NP**-hard optimization problems?
2. For a minimization problem, why is the ratio C/C^* instead of C^*/C ?
3. What does it mean for an algorithm to be a 2-approximation?
4. Why do the edges chosen by APPROX-VERTEX-COVER form a matching?
5. In the proof of APPROX-VERTEX-COVER, why must every vertex cover contain at least $|A|$ vertices?
6. Why is the triangle inequality necessary for the metric TSP approximation?
7. Why does deleting one edge from an optimal TSP tour give an upper bound on the MST cost?
8. Why does the no-approximation result for general TSP depend on $\mathbf{P} \neq \mathbf{NP}$?